CRIMEVISIONAI

Code Layout, Readability and Reusability

Folder 5 | Code quality review based on reported architecture | Prepared 23 October 2026

Project: CrimeVisionAI — Multimodal Forensic Evidence Analysis and Investigation Support System

Document status: Project-specific draft based on the implementation information currently available. Verify placeholders and run-dependent results before formal submission.

1. Current structure overview

Known high-level structure includes a Flask app factory, run.py entry point, Jinja2 frontend, CSS/JavaScript assets, API routes, SQLite database file and local evidence upload directories. Exact module paths should be copied from the current source tree during final code review; this ZIP does not include the application source code.

2. Code layout checklist

#	Quality parameter	Assessment	Remarks / follow-up
1	Consistent indentation	Verify in source	Run formatter/linter or inspect representative modules.
2	Logical file structure	Partially evidenced	App factory and frontend/backend separation reported; confirm exact layout.
3	Meaningful variable names	Verify in source	Review route and analysis function naming.
4	Descriptive functions	Verify in source	Ensure each modality pipeline has clear responsibilities.
5	Comments / docstrings	Verify in source	Document non-obvious model preprocessing and response schemas.
6	Modular design	Partially evidenced	App factory and separate modality routes reported; avoid claiming full modularity without review.
7	No redundant / dead code	Not verified	Run static review; remove unused imports, duplicate handlers and stale experiments.
8	Error handling	Partially evidenced / verify	Test invalid uploads, missing models, API errors and DB failures.

3. Reusable components / modules

Component	Language / technology	Reuse opportunity	Assessment
Case/evidence API helpers	Python / Flask / sqlite3	Shared case lookup, evidence lookup and JSON response patterns	Potentially high; verify implementation.
Audio analysis pipeline	Python + Whisper + Groq	Reusable for audio evidence across cases	High within current application.
Image analysis pipeline	Python + RT-DETR + CLIP + BLIP + Groq	Reusable image analysis component	High within current application.
Video frame pipeline	Python + video processing + vision models	Reuses image pipeline for sampled frames	Medium/high; confirm shared functions.
Frontend API helpers	JavaScript	Centralize fetch, loading and error handling	Potentially medium; inspect source.
DB access helpers	Python sqlite3	Centralize queries, transactions and schema handling	Potentially high; inspect source.

4. Recommended engineering checks

• Add consistent structured logging with request/evidence identifiers, avoiding sensitive evidence content in logs. • Validate extension and actual file type, enforce file size, and use safe generated filenames. • Centralize DB connection/transaction handling and schema initialization. • Separate raw model observations from LLM interpretation in response schemas. • Use configuration via environment variables and never commit secrets. • Add unit tests for pure helpers and integration tests for routes. • Disable debug mode outside local development.

5. Overall assessment

Current architecture has reasonable prototype-level separation by application, frontend, persistence and modality-specific analysis. A definitive code-quality score cannot be assigned without source inspection. The checklist deliberately marks unverified properties as requiring review rather than awarding unsupported scores.